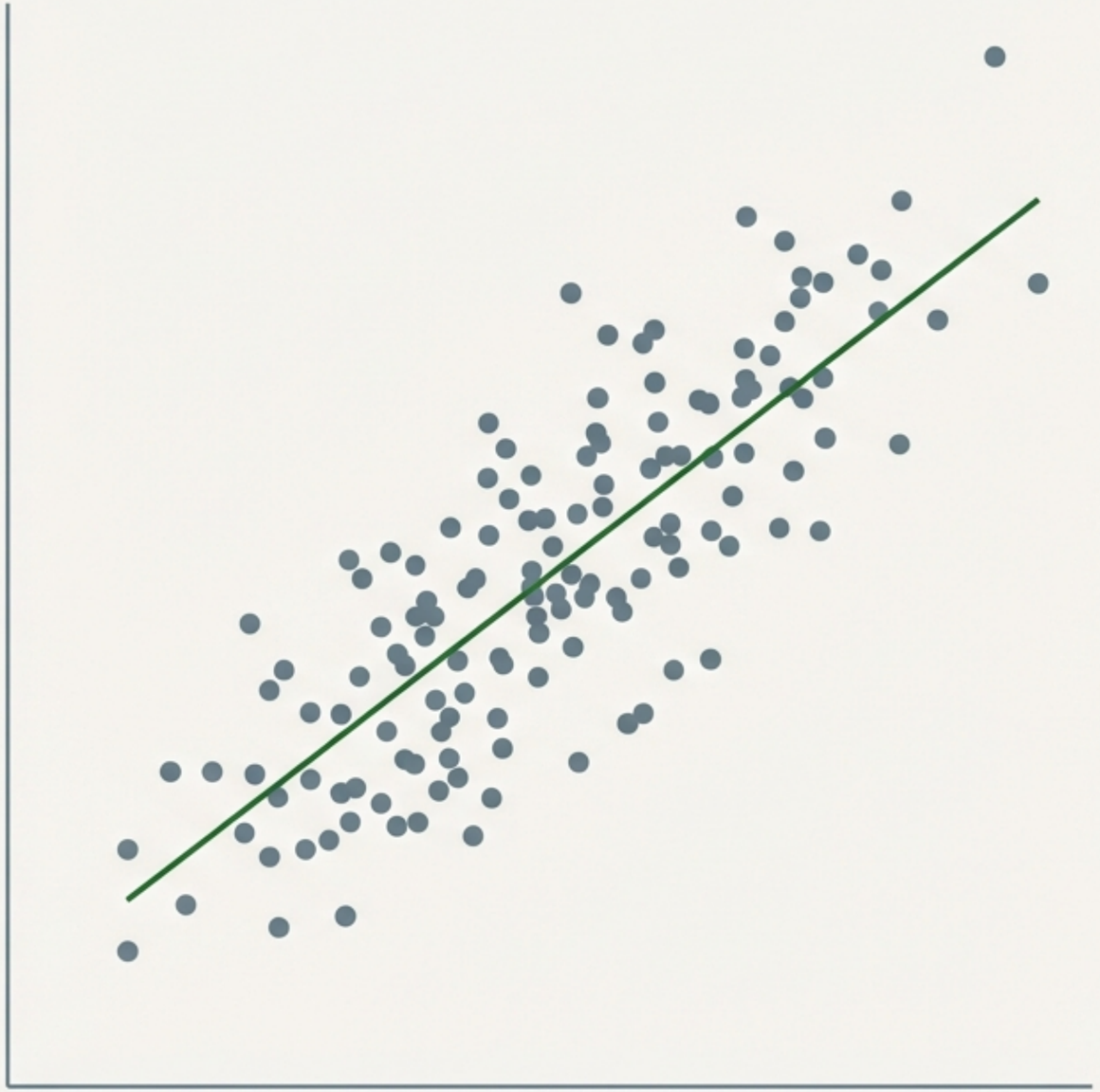
Measuring Data Relationships: Covariance & Correlation

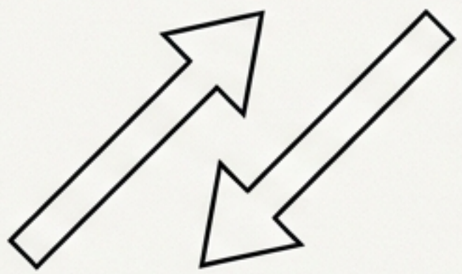
A Guide to Understanding Feature Interactions in Data Science



Understanding how features interact is the core of Exploratory Data Analysis (EDA).

To understand data, we must answer three fundamental questions:

Direction



Do variables move together?

Strength

How strong is the link?

Shape

Is the relationship linear or complex?

The Toolkit:

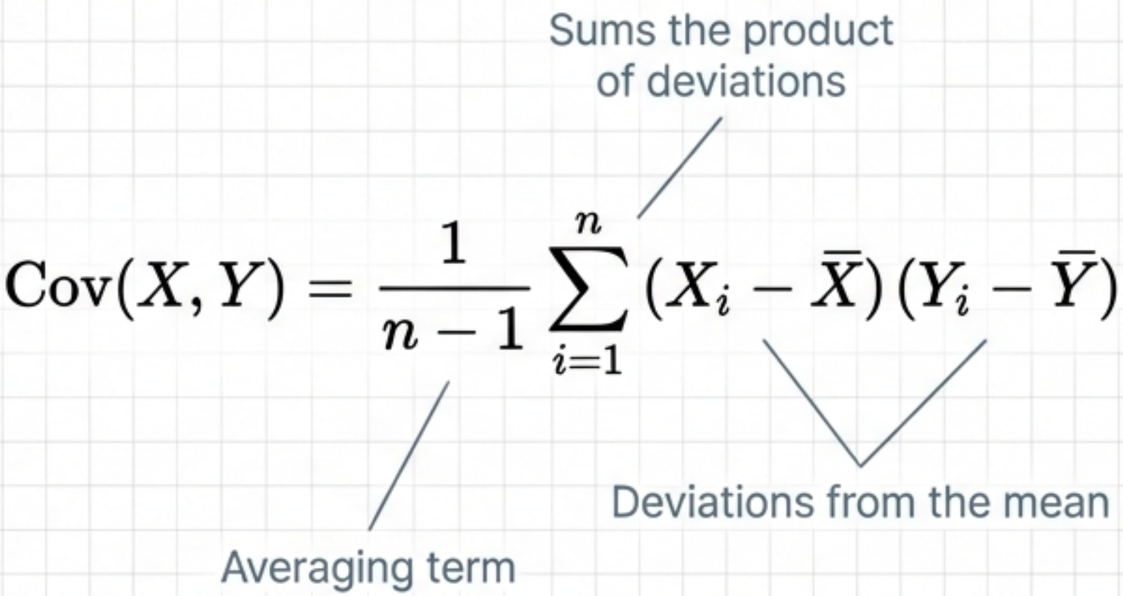
1. Covariance: The foundational measure of direction.
2. Pearson Correlation: The standardised measure of linear strength.
3. Spearman Rank Correlation: The robust measure for non-linear or ranked relationships.

Part I: Covariance (The Foundation)

Covariance measures the directional relationship between two variables.

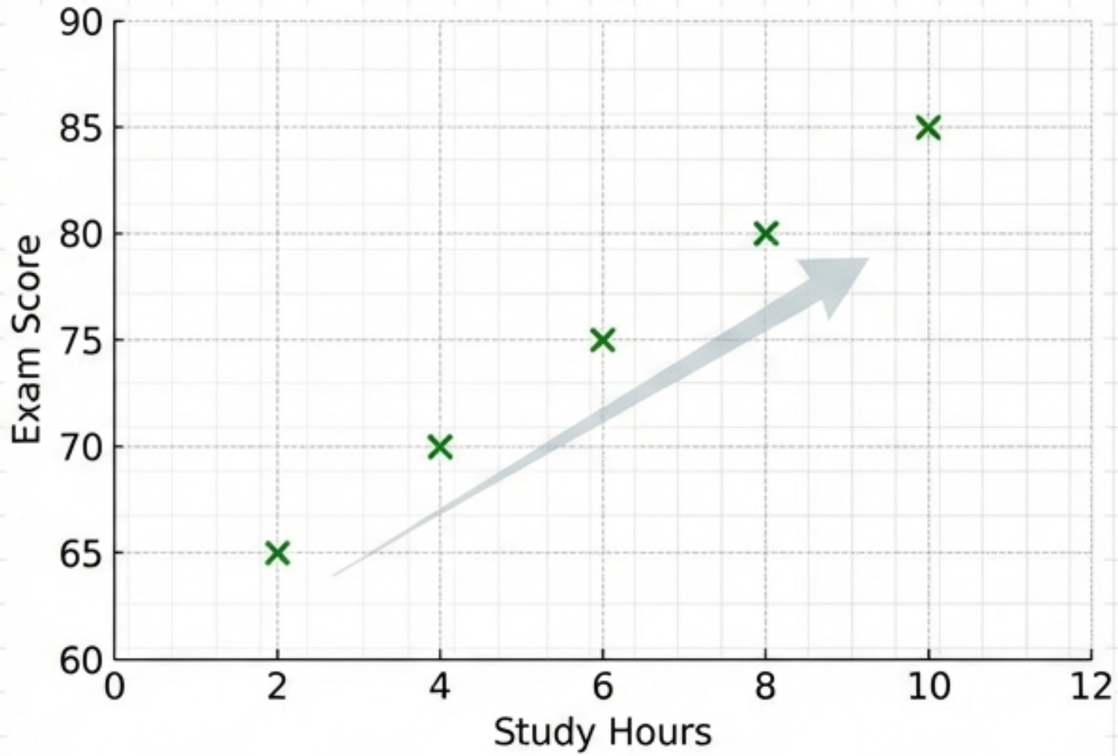
The Core Logic:

- **Positive Covariance:** Both variables increase together (X↑, Y↑).
- **Negative Covariance:** One increases, one decreases (X↑, Y↓).
- **Zero/Near-Zero:** No discernible pattern.



Applying Covariance to real data: Study Hours vs. Exam Scores

Student	Study Hours (X)	Exam Score (Y)
1	2	65
2	4	70
3	6	75
4	8	80
5	10	85



Positive Covariance: As X increases, Y increases.

Calculating Covariance in Python

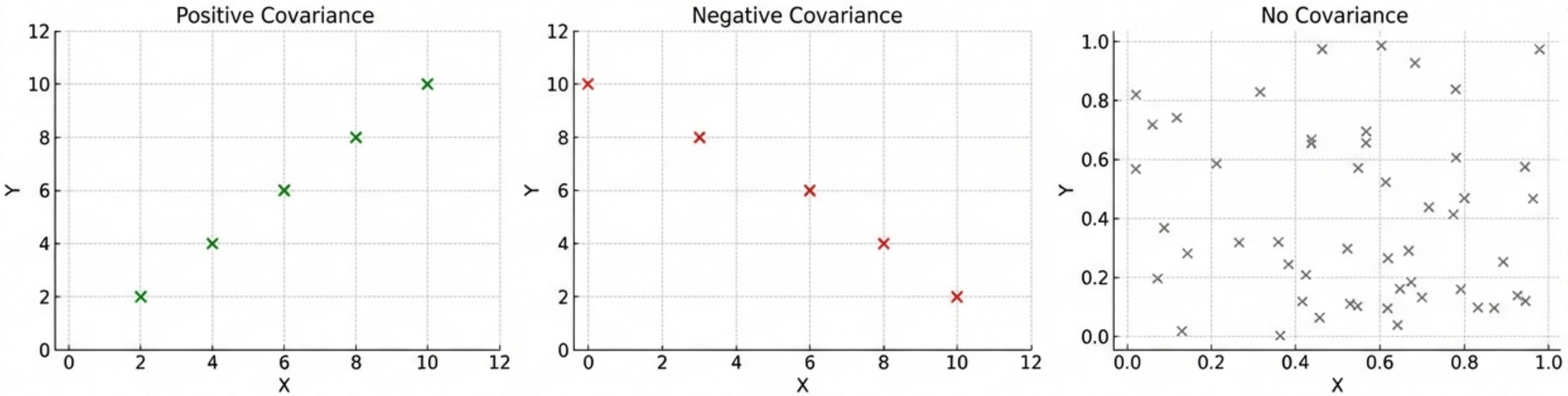
```
import numpy as np

# Data
study_hours = np.array([2, 4, 6, 8, 10])
exam_scores = np.array([65, 70, 75, 80, 85])

# Calculate Covariance
cov_matrix = np.cov(study_hours, exam_scores)
print(cov_matrix)
```

```
[[ 10.   25. ]
 [ 25.   62.5]]
```

Covariance = 25.0. The positive value confirms the positive relationship.

The limitation: Covariance values are difficult to interpret.

Covariance is NOT unit-free. The magnitude depends on the scale of the data.



Why is a covariance of '25.0' problematic?

If we measured study time in minutes instead of hours, the covariance would jump to 1,500 despite the relationship being identical. We need a standardised score.

Part II: Pearson Correlation (The Standard)

Pearson Correlation creates a standardised, interpretable score.

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y}$$

Direction
(Raw Covariance)

Standardisation (Product of Standard Deviations)

By dividing covariance by the standard deviations, the result is normalised to a range of -1 to +1.

Technical Editorial

Reading the Pearson Coefficient (r).

$$-1 \leq r \leq 1$$

- Use +1 for lockstep increases.
- Use –1 for lockstep inverse movements.
- Use 0 for random noise.

Primary Use Case: Feature selection in Machine Learning.

Technical Editorial

Pearson Correlation in Python.

Code Block

```python
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10]) # Strong positive

# Calculate Pearson Correlation
correlation = np.corrcoef(x, y)[0, 1]

print(f'{correlation:.2f}')
```

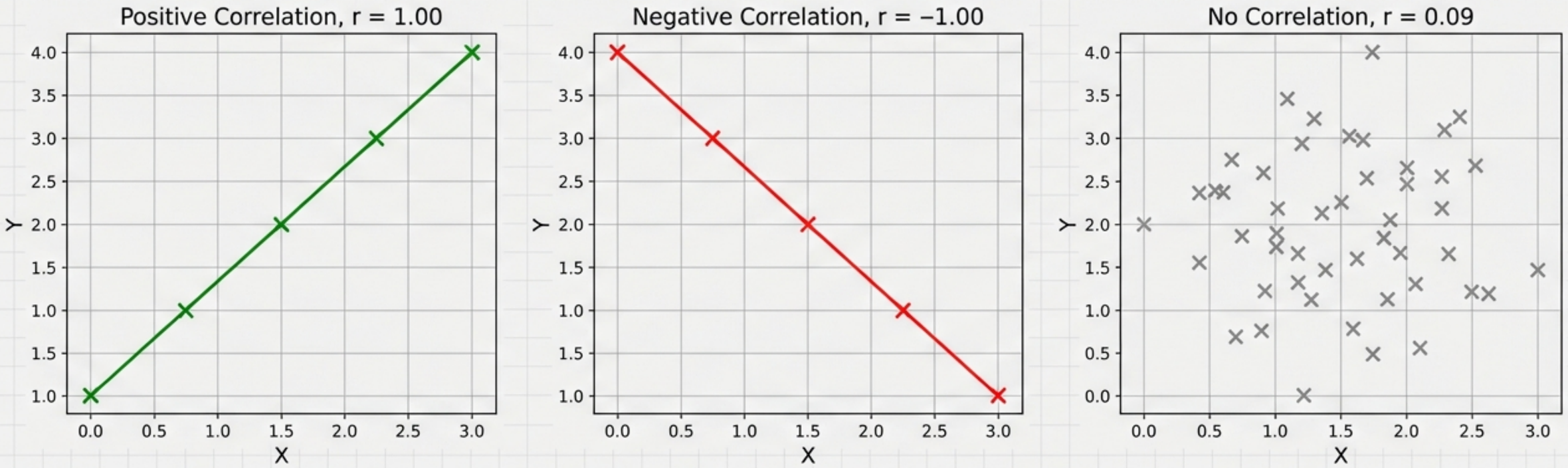
Output & Interpretation

1.00

Interpretation:
A value of 1.00 indicates a Perfect Positive Linear Relationship.

Key Takeaway: Unlike Covariance, this '1.00' is universal. It means the same thing regardless of whether the variables are measured in millions or decimals.

Visualising Pearson Correlation (r)



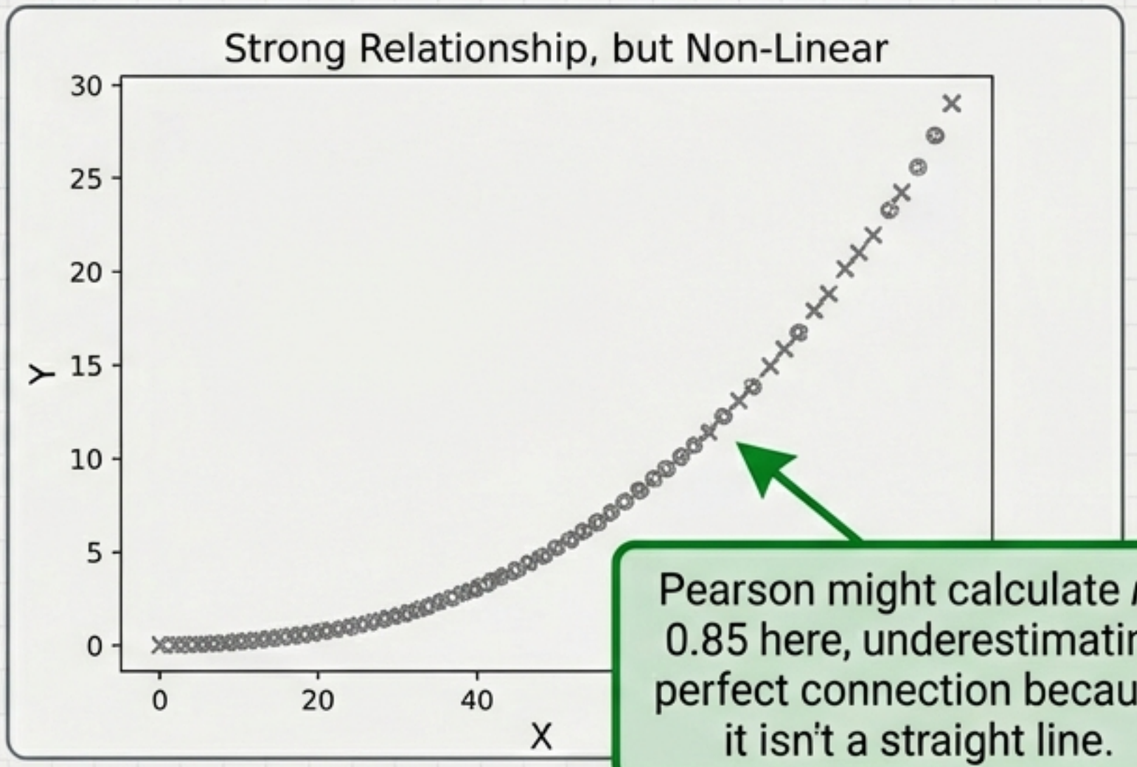
Pearson measures linearity. It asks:
“How well do these points fit a straight line?”

Technical Editorial

The limitation: Pearson assumes a linear relationship.

Pearson is powerful, but rigid. It fails when:

1. Data is **monotonic** (consistent trend) but **curved** (e.g., exponential growth).
2. Data is **Ordinal** (ranked categories like '1st Place', '2nd Place').



Solution: Non-Parametric Statistics

Technical Editorial

Part III: Spearman Rank Correlation

The non-parametric alternative for non-linear or ranked data.

Definition

Spearman measures the strength and direction of a monotonic relationship based on ranks rather than raw values.

Formula

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Where:

- d_i = difference between ranks of x_i and y_i
- n = number of observations

Checklist

- When to use Spearman (ρ):
- ✅ Variables are Ordinal (Ranked).
- ✅ Data is non-linear but monotonic (consistently increasing or decreasing).
- ❌ If data is linear and continuous, stick to Pearson.

Scenario: Comparing Academic Rank vs. Sports Rank

We want to check if being top of the class implies being bottom in sports (an inverse relationship).

```
from scipy.stats import spearmanr

# Ranks (1 is top rank)
academic = [1, 2, 3, 4, 5, 6]
sports   = [6, 5, 4, 3, 2, 1]

rho, p_val = spearmanr(academic, sports)
print(f'{rho:.2f}')
```

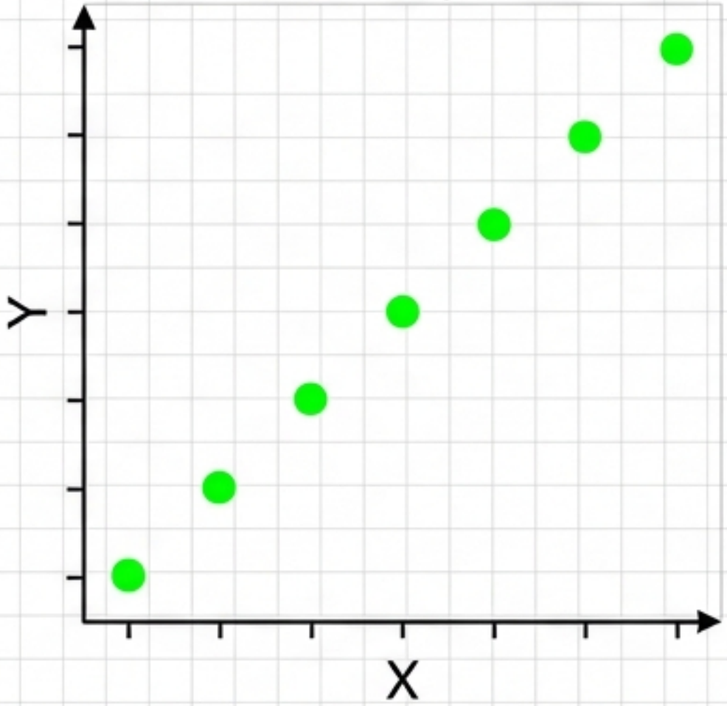
```
-1.00
```

Result: **-1.00**. This confirms a perfect negative monotonic relationship. As academic rank goes up (worse), sports rank goes down (better).

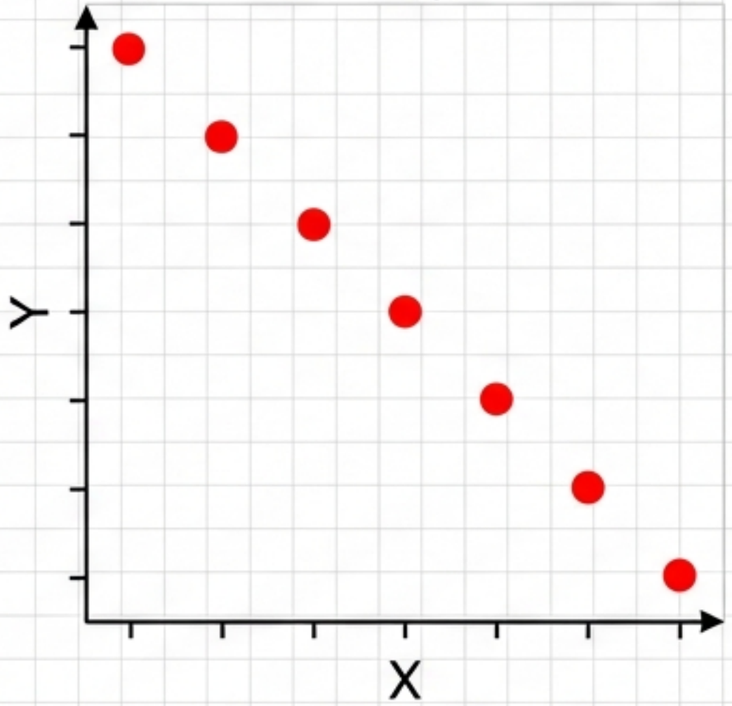
Technical Editorial

Visualising Spearman Rank Correlation (ρ).

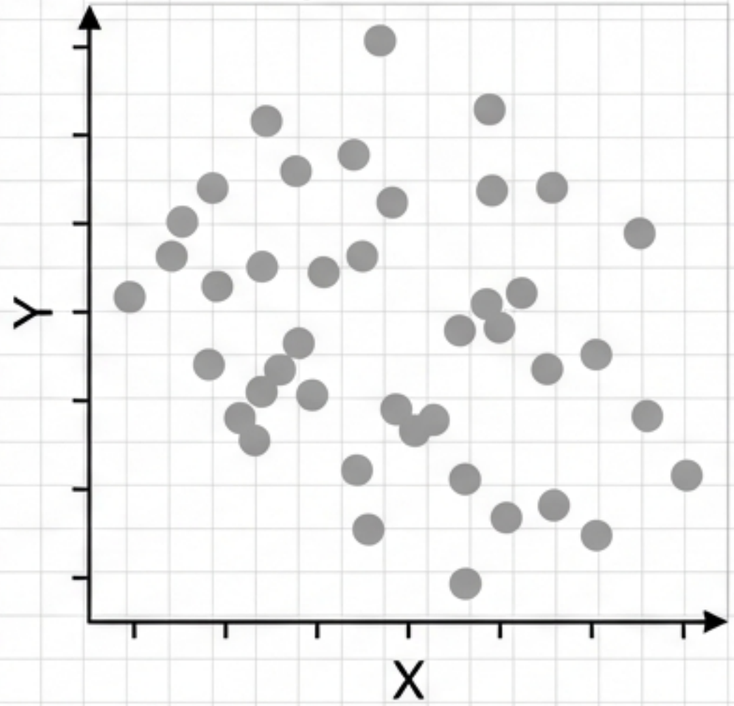
Perfect Positive Rank Correlation, $\rho = 1.00$



Perfect Negative Rank Correlation, $\rho = -1.00$



No Rank Correlation, $\rho = -0.30$



Monotonicity explained: The raw data points don't need to form a straight line. As long as the rank order is preserved (never reversing direction), Spearman captures the relationship perfectly.

Summary: Choosing the Right Metric

Metric	Use Case	Data Type	Pros/Cons
Covariance	Determine Direction Only	Continuous	Hard to interpret magnitude (Not unit-free).
Pearson (*r*)	Linear Strength	Continuous & Linear	Standardised (-1 to 1). Default for EDA.
Spearman (*ρ*)	Monotonic Strength	Ordinal / Non-Linear	Robust to outliers and curves.

Cov: Covariance (Direction) | ***r***: Pearson Coeff. (Linear) | ***ρ***: Spearman Coeff. (Rank) | ***σ***: Standard Deviation